System Architecture & *AI Agent* Systems.

How systems are structured, how agents are built, and how to wire them together.

The arc.

Part 01 · 30 min — System architecture fundamentals.

Part 02 · 20 min — AI agent architecture.

Part 03 · 10 min — Agent orchestration patterns.

What is *system architecture*?

The high-level structure of a software system.

IT INCLUDES

Major components — services, modules, databases.

Relationships and dependencies.

Communication mechanisms — APIs, queues, events.

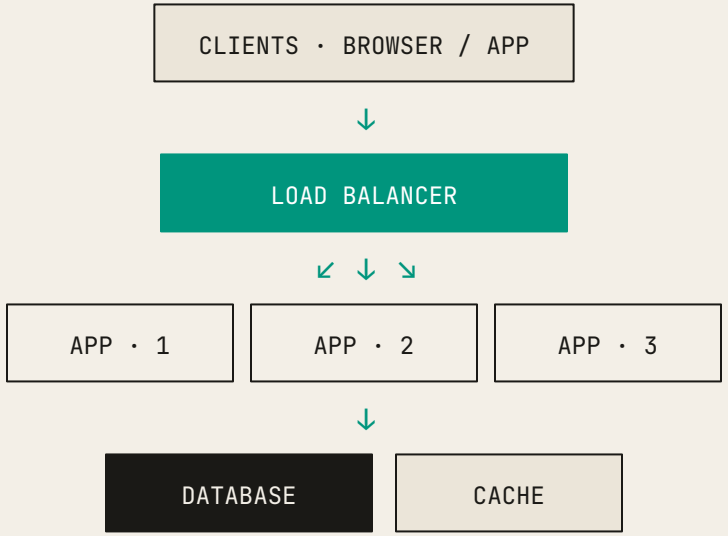
Infrastructure — servers, networks, storage.

IT IS NOT

Implementation details — algorithms, data structures.

Class methods or variable names.

Code organization within a single file.



A typical system architecture.

Why architecture matters.

I.

Bridges requirements to code.

Requirements say *what* . Architecture says *how* . Implementation executes.

II.

Communicates across stakeholders.

Developers, architects, managers, testers — one shared diagram.

III.

Lasts.

Early decisions are expensive to reverse. Rearchitecting takes months or years. Choose wisely upfront.

Real impact: *Netflix.*

2000s

Started as a monolithic application — reasonable for the scale.

2008 · 2015

Re Architected to microservices over *seven years.*

Rebuilt deployment infrastructure.

Reorganized the entire company.

Migrated service by service.

Outcome

Independent scaling. Team autonomy. Multiple daily deployments — at production scale, thousands per day.

Architecture changes are expensive. Choose carefully and evolve deliberately.

Core design principles.

I · DECOMPOSITION

Divide and conquer.

Break a complex system into focused modules, each with one clear responsibility.

`e-commerce → user · catalog · cart · payment · shipping`

II · ABSTRACTION

Hide the how, expose the what.

Modules depend on interfaces, not implementations. Swapping a database doesn't ripple through the app.

III · COUPLING & COHESION

Low coupling. High cohesion.

Modules depend on each other as little as possible. Everything inside a module belongs together.

Quality factors — the *-ilities*.

FACTOR	DEFINITION	NOTE
Scalability	Handle increased load — vertical (bigger box) or horizontal (more boxes).	Horizontal usually wins on the web.
Performance	Throughput (req / s) and latency (ms / req).	Two different numbers — measure both.
Availability	Percent of time the system is operational.	Five nines ≈ 5.26 min down / year.
Reliability	System produces correct results under stated conditions.	Available ≠ correct.
Maintainability	Ease of fixing bugs and adding features.	A team property as much as a code property.

Architecture is about *trade-offs.*

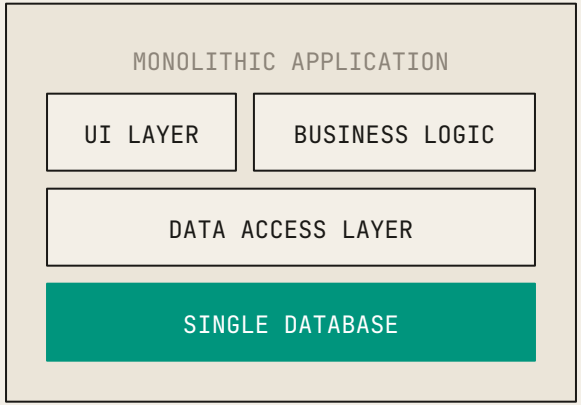
You can't optimize everything simultaneously.

OPTIMIZE FOR	TRADE-OFF AGAINST	EXAMPLE
Performance	`Cost`	More servers, faster databases — money.
Availability	`Cost`	Redundant systems double the bill.
Scalability	`Complexity`	Microservices scale but are harder to debug.
Security	`Performance`	Encryption and auth checks add latency.
Flexibility	`Performance`	Abstraction layers add overhead.

Monolithic architecture.

All functionality in a single codebase, single deployment.

PROS	Simple to develop, test, and deploy. Fast in-process calls. Easy debugging.
CONS	Hard to scale parts independently. Codebase grows hard to hold. Technology lock-in.
USE WHEN	Small teams (<10), startups, MVPs, simple domains.

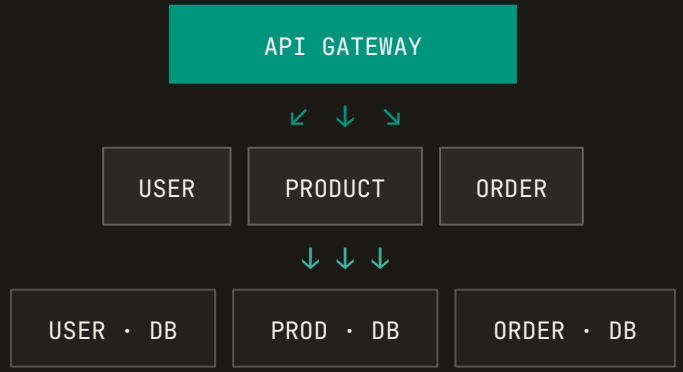


One codebase. One deployment.

Microservices architecture.

System split into small, independent services.

PROS	Independent scaling. Team autonomy. Fault isolation. Tech flexibility.
CONS	Distributed complexity. Network overhead. Data consistency. Operational load.
USE WHEN	Large teams (>20). Need independent scaling. Different teams, different expertise.



Layered architecture.

System organized into horizontal layers.

Each layer talks only to the layer directly below it. Separation of concerns; layers can be swapped independently.

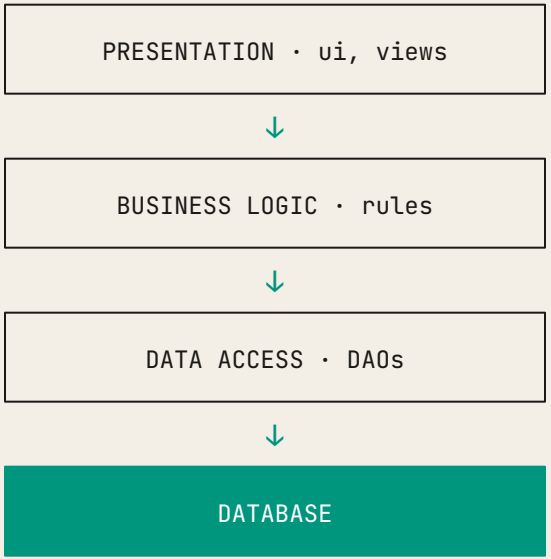
APPLIED TO AI SYSTEMS

UI — chat interface

Orchestration — agent logic, tool coordination

Model & tool — LLM API, RAG, external APIs

Storage — vector DB, SQL, NoSQL

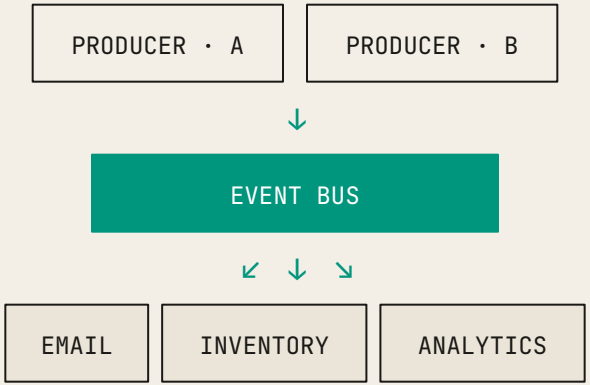


Event-driven architecture.

Components communicate through asynchronous events.

EVENT	Something happened — `UserRegistered`.
PRODUCER	Publishes events.
CONSUMER	Subscribes and reacts.
BUS	Routes events between them.

Pros — loose coupling, async, easy to extend. *Cons* — debugging spans services, eventual consistency.



Client-server architecture.

Clients request services. Servers provide them. Foundation for most modern systems.

THIN CLIENT

Minimal logic. Just display.

Server handles all business logic. Easy updates, cross-platform, requires connection.

`Gmail · Google Docs · most web apps`

FAT CLIENT

Significant local processing.

Client processes locally, syncs to server. Works offline, better local performance, harder to update.

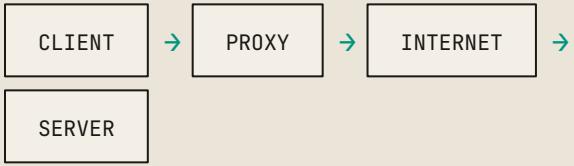
`IDEs · video editors · desktop tools`

Infrastructure: *proxy servers.*

Intermediaries between client and destination.

FORWARD PROXY · CLIENT-SIDE

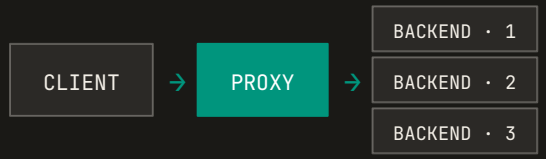
Hides the client from the internet.



Anonymity, content filtering, caching. Corporate proxy, VPN.

REVERSE PROXY · SERVER-SIDE

Hides the servers from the internet.



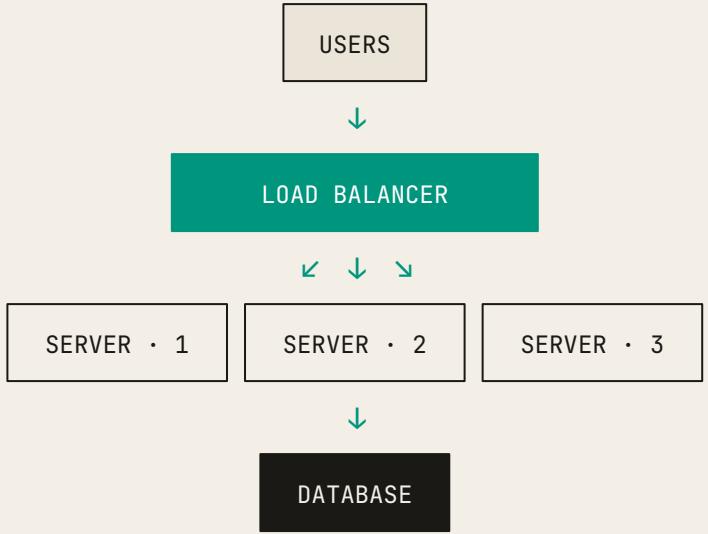
Load balancing, SSL (Secure Sockets Layer) termination, security, caching. Nginx, HAProxy, CloudFlare.

Load balancing & caching.

I · LOAD BALANCING

Distribute requests across multiple servers.

- `round robin` — alternate servers in order.
- `least connections` — pick the least-busy server.
- `ip hash` — same user always lands on same server.



II · CACHING

Store frequently accessed data closer to the request.

Browser cache → CDN (Content Delivery Network) cache→ application cache like Redis → database cache

CDN & message queues.

CDN · CONTENT DELIVERY NETWORK

Cache content close to the user.

Lower latency, lower bandwidth costs, DDoS absorption. CloudFlare, Akamai, CloudFront.

A Content Delivery Network (CDN) is a geographically distributed group of servers that caches content—such as images, videos, and website files—close to end-users

MESSAGE QUEUES

Asynchronous communication.

Benefits: Decoupling, load leveling, reliability. Background jobs that the user shouldn't wait on. Kafka, RabbitMQ, SQS.

A message queue is a component in software architecture that facilitates asynchronous, service-to-service communication by storing messages in a buffer until they are processed

A classic example is e-commerce order processing: the frontend quickly sends an order to a queue, while backend workers process payments and send confirmation emails at their own pace.

Rate limiting.

Restrict the number of requests in a time window.

WHY YOU NEED IT

Prevent abuse and accidental DDoS.

Fair resource allocation across users.

Cost control — agents calling paid APIs.

Protect downstream services.

AI TIER EXAMPLE

A bug that puts an agent in a loop, calling an LLM at `$0.03` per request, is how an overnight bill becomes five figures.

`Free` — 10 LLM req / min `Pro` — 100 LLM req / min

CONWAY'S LAW · 1967

“Organizations design systems that mirror their communication structures.”

— MELVIN CONWAY

TEAM STRUCTURE	RESULTING ARCHITECTURE
User team	User service
Payments team	Payment service
Products team	Product service
Orders team	Order service

Your software architecture will look like your team structure, whether you intend it or not. Four teams that don't talk to each other will produce four modules that don't integrate well.

Inverse Conway maneuver.

Intentionally design team structure to achieve the architecture you want.

TRADITIONAL

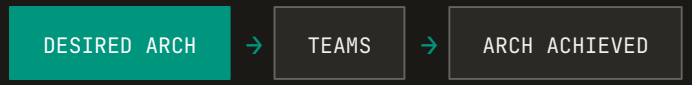
Org → Architecture.

Architecture is whatever falls out of your accidental org chart.

INVERSE CONWAY

Architecture → Org.

Want microservices? Build small autonomous full-stack teams. They build the architecture you wanted.



Start *simple.*

I · DEFAULT

Start with a monolith.

Faster to build. Easier to change. Good for small teams, MVPs, unclear requirements. You can always split it later.

II · MIGRATE WHEN

Team grows beyond 15–20 people.
Service boundaries become clear.
Different parts need very different scaling.
Different parts evolve at very different rates.

ANTI-PATTERN

Resume-driven development.

Choosing microservices because they look good on a CV. Result: complexity without the benefits.

You're not Google. You're not Netflix. They started with monoliths too.

Part 01 — summary.

QUALITY FACTORS

Scalability · Performance · Availability · Reliability · Maintainability. They trade off.

PATTERNS

Monolith — simple, small teams.
Microservices — complex, large teams.
Layered — separation of concerns.
Event-driven — loose coupling.
Client-server — foundation of the web.

INFRASTRUCTURE

Proxy — forward and reverse.
Load balancer.
Caching at every level.
CDN — content close to users.
Message queues — async work.
Rate limiting — abuse and cost.

ORGANIZATION

Conway's law — architecture mirrors teams.
Inverse Conway — design teams to get architecture.
Start simple. Evolve as needed.

Architecture is not just technical – it's organizational.

PART — 02

AI Agent *architecture.*

Model. Tools. Instructions. Memory. Guardrails. The anatomy of a working agent.

When to build an agent?

BUILD ONE WHEN

Task takes multiple steps or decisions.

Needs to use external tools — APIs, databases, search.

Requires adaptive reasoning — can't predict all paths.

DON'T BUILD ONE WHEN

A single LLM call is enough.

The task is simple classification or generation.

A deterministic workflow does the job better.

THE DIVIDING LINE

Autonomy with tools. The agent decides what to do next based on what it has seen so far.

Three core components.

I · MODEL

The brain.

The LLM that does the reasoning. Decides next action, interprets tool results, generates the final answer.

`GPT-4 · Claude · Gemini`

II · TOOLS

The hands.

Functions the agent can call. Transforms a text generator into a task executor.

```
web_search() · sql_query() · send_email()
```

III · INSTRUCTIONS

The goals.

System prompt — role, goals, constraints. Where business rules live.

"You are a research assistant who..."

All three required. Model alone is a chatbot. Add tools and you have an agent. Add instructions and it's useful.

Agent memory systems.

SHORT-TERM · SESSION STATE

What happened this session.

Last 10 messages.

Tool results from the current task.

Current task context.

`dev: in-memory · prod: Redis (horizontal scaling)`

LONG-TERM · KNOWLEDGE BASE

What persists across sessions.

User preferences.

Past conversations — searchable via RAG.

Learned facts and company knowledge.

`vector DB · SQL · knowledge graph`

Context windows are finite — Claude 200k, GPT-4 128k. Be selective.

RAG

I · INDEX

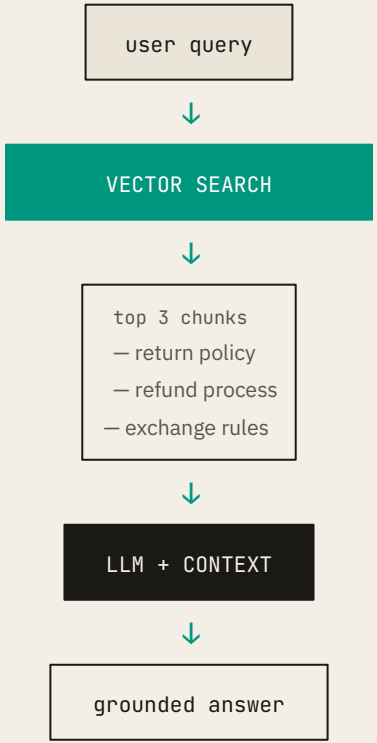
Chunk documents → embed → store in vector DB. Done once.

II · RETRIEVE

Embed the query → find most similar chunks → top-k relevant snippets.

III · GENERATE

Hand the LLM the question + retrieved chunks. It answers using that context.



Context window management

- **Context Summarization:** Compresses older conversational turns into a concise summary to free up space while retaining context.
- **Sliding Window/Truncation:** Removes the oldest messages in a conversation as new ones arrive to keep the total token count within limits.
- **RAG (Retrieval-Augmented Generation):** Instead of feeding all documents into the prompt, store them in a database and only retrieve relevant segments to add to the context.
- **Observation Masking:** Hides noisy or irrelevant tool outputs (like long logs) from previous agent steps to save tokens.

Common Challenges

- **Lost in the Middle:** Models tend to be better at recalling information at the very beginning or end of a prompt, often missing details placed in the middle.
- **Cost Control:** Larger context windows allow for more data but significantly increase the token cost per request.
- **Latency:** Processing extremely long context windows (e.g., 1M+ tokens) can lead to slower response times.

Agent tools & *MCP.*

WHAT IS A TOOL?

A function the agent can ask you to call. Input — parameters. Output — structured result.

```
get_weather(city="Tokyo") → {temp: 72, cond: "sunny"}
```

MODEL CONTEXT PROTOCOL

Anthropic's open standard for tool interfaces. Like USB — write the tool once, any compliant model can use it.

THE BLOAT PROBLEM

Too many tools → wrong tool selection.

5 tools	~95% correct
20 tools	~75% correct
50 tools	~60% correct

Aim for under ten tools per agent.

Mitigating tool bloat.

I · SIMPLIFY

Limit tool complexity.

Fewer than 5 parameters. Use enums, not free-text strings. The agent shouldn't have to write `"urgent!!!"`.

```
priority: "high" | "med" | "low"
```

II · SEGMENT

Granular toolsets.

Customer agent gets ticket tools. Admin agent gets database tools. Don't pile them all on one agent.

III · DISCLOSE

Progressive disclosure.

Start with meta-tools. The agent searches for the tools it needs and loads them on demand.

Agent guardrails.

Layered defense — multiple checks, each catching a different class of problem.

i · Relevance	Filter off-topic queries. Customer support agent rejects "tell me a joke."
ii · Safety	Block harmful or jailbreak attempts.
iii · PII filter	Detect and redact personal data — SSN, credit cards.
iv · Moderation	Toxicity, bias, policy violations on input and output.
v · Tool guards	Validate high-risk tool calls before execution. Require approval for destructive ops.
vi · Rules-based	Hard business rules that override the model. "Never refund > $500 without approval."
vii · Output	Validate response format and quality before returning to the user.

Agent runtime & the run loop.

TWO RUNTIMES

AGENT Your code — orchestration, tool execution, memory.

MODEL LLM provider — inference, GPUs.

THE LOOP

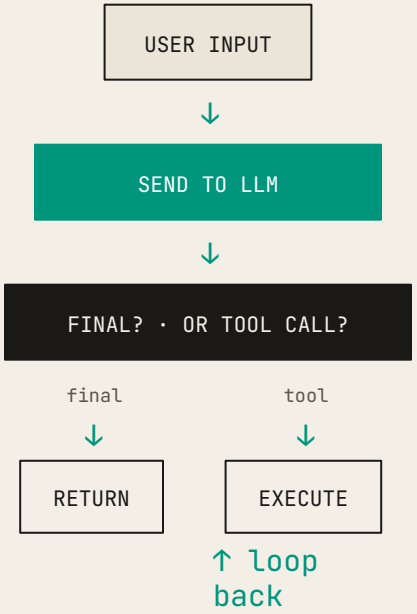
Receive user input.

Send to model.

Model returns: final answer, or tool call.

Execute tool. Feed result back.

Repeat until done — or max iterations.



Always cap iterations. Without one, an agent can loop until your budget runs out.

Human in the loop.

WHEN TO INVOLVE A HUMAN

High-risk actions — delete data, financial transactions.

Agent confidence below threshold.

After N failed retries.

Compliance-mandated approval.

IMPLEMENTATION

Slack/email approval workflows.

Escalation queues for human review.

Audit logs of every agent action.

RISK-TIERED EXAMPLE · CUSTOMER SUPPORT

`autonomous`	Answer FAQs · check order status
`audit`	Schedule callback · update address
`approve`	Issue refund · close account

Start with more oversight. Automate boring patterns once you've watched humans always approve them.

Part 02 — summary.

ANATOMY

Three components — model, tools, instructions.
All three required.

TOOLS

MCP — standard interface.
Tool bloat is real (< 10 per agent).
Simplify · segment · disclose.

RUNTIME

Run loop — input, model, tool, repeat.
Cap iterations.
Multiple LLM calls per query.

MEMORY

Short-term — session state, Redis in prod.
Long-term — RAG, vector DB, SQL.

SAFETY

Seven guardrails, layered.
HITL for high-risk actions.

Agents are powerful — and that's why they need careful design.

PART — 03

Agent *orchestration.*

When to use a single agent and when you need many. Seven patterns and how to choose between them.

Single-agent systems.

One agent handles the whole task.

REACT · REASON AND ACT

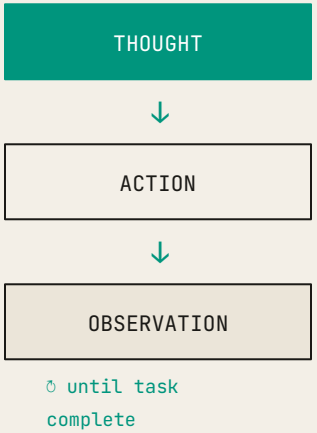
`thought` — what should I do next?

`action` — call a tool, or final answer.

`observe` — feed the result back.

Pros — flexible, simple. *Cons* — latency and cost grow with loops.

When in doubt — build this first.



Sequential pattern.

A linear pipeline. Agent A → Agent B → Agent C, always in that order.

PROS — Low latency. Predictable. Easy to debug.

CONS — Rigid. Can't skip steps. One failure breaks the chain.

USE WHEN — Predefined steps, no dynamic routing needed.

From an engineering perspective, this is just function composition:

```
output = c(b(a(input)))
```

examples — ETL pipelines, document processing, content workflows

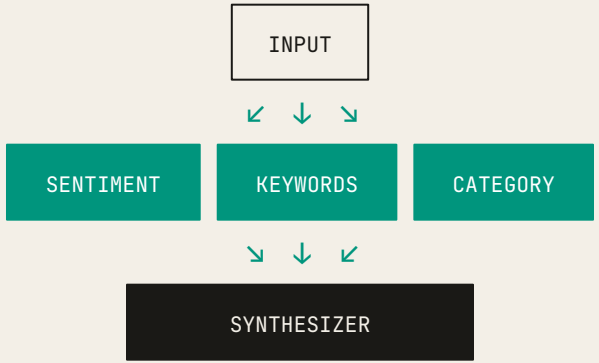
Parallel pattern.

Independent sub-tasks, run concurrently.

`customer review` — run sentiment, keyword extraction, and category classification at the same time. Synthesizer combines.

PROS Wall-clock = slowest agent, not the sum.

CONS Higher cost — multiple concurrent calls. Synthesizer must handle partial failures.



ReAct — deep dive.

Plan a 3-day trip to Tokyo.

`loop 1` — thought: weather first. action: `get_weather`. obs: sunny days 1–2, rainy day 3.

`loop 2` — thought: outdoor for sunny. action: `search("outdoor attractions")`.

`loop 3` — thought: indoor for rainy. action: `search("museums")`.

`loop 4` — thought: enough. final: a 3-day plan.

WHEN TO USE

Open-ended tasks needing autonomous decisions.

Adapts when results surprise. More human-like reasoning. Default in LangChain, AutoGPT, most frameworks.

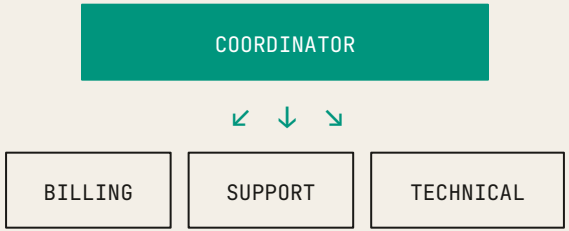
Always cap iterations — usually 5 to 10 — or you'll loop forever.

Coordinator pattern.

Router agent classifies. Specialist agent handles.

Like a receptionist directing you to the right department.

PROS	Specialized agents are sharper. Tool isolation. Easier to debug.
CONS	Extra coordinator call. Routing errors are their own failure mode.
USE WHEN	Varied inputs map to different specialists — customer support, enterprise search.



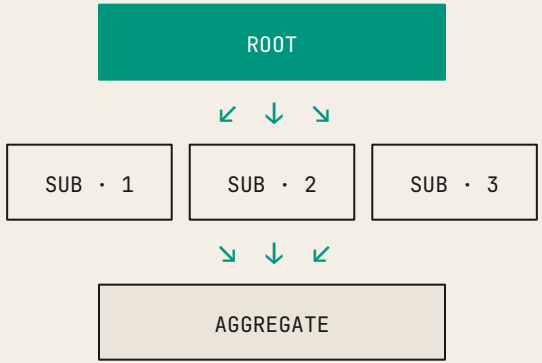
Hierarchical decomposition.

A root agent breaks a complex task into sub-tasks and spawns sub-agents.

`analyze top 5 competitors` → root spawns five research sub-agents, aggregates results into a final report.

PROS Handles genuinely complex tasks. Natural decomposition. Parallelizable.

CONS Very expensive. High latency. Coordination overhead. Tendency to over-decompose.



Hierarchical vs *Parallel.*

ASPECT	HIERARCHICAL DECOMPOSITION	PARALLEL PATTERN
Input	One complex task.	One piece of data.
Purpose	Break a big task into smaller tasks.	Analyze the same data from multiple angles.
Task type	Different sub-tasks.	Same analysis, different perspectives.
Structure	Tree — can be multi-level.	Flat — one level.
Agents	Do different work.	Do the same kind of work on the same input.
Execution	Sequential or parallel.	Always concurrent.
Example	`research 5 competitors` → 5 distinct tasks.	`analyze review` → sentiment + keywords + category.
Latency	Very high — sequential levels.	Medium — concurrent + synthesis.

Review & critique.

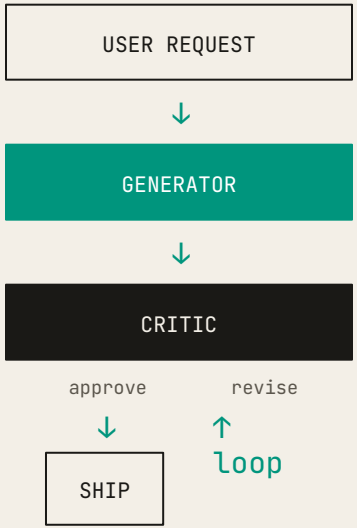
Generator produces. Critic evaluates. Loop until approved.

Used where errors are expensive — legal documents, security-sensitive code, medical content.

Constitutional AI is essentially this pattern applied to safety alignment.

PROS Higher quality. Specialized critic. Catches errors before users see them.

CONS Slow. Expensive. Cap iterations or it loops.



Human-in-the-loop.

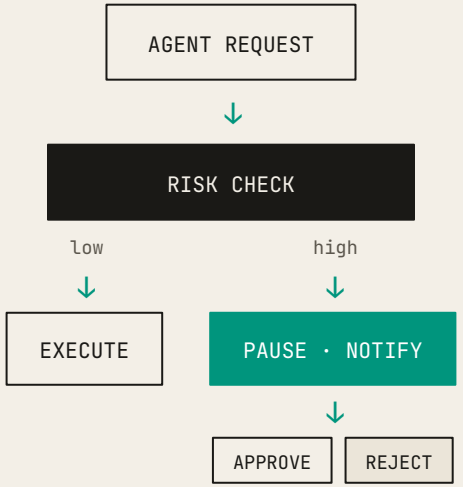
A pause-for-approval node anywhere in the graph.

Refunds over a threshold.

Account deletion.

Production deployment.

Anything irreversible or compliance-bound.



IMPLEMENTATION

Slack/email approvals. Dashboards for pending items. Audit trail of every decision.

Choosing the right pattern.

IF YOUR TASK IS…	USE THIS PATTERN
Predefined linear steps	`Sequential`
Independent sub-tasks needing speed	`Parallel`
Open-ended, single domain	`ReAct · single agent`
Varied inputs across domains	`Coordinator`
Genuinely complex, naturally decomposable	`Hierarchical`
Quality-critical output	`Review & critique`
High stakes	`+ HITL on top`

Patterns compose — coordinator + HITL is normal.

Start *simple.*

COMPLEXITY LADDER

Single agent — ReAct. Default.

Tool optimization — bloat mitigation.

Coordinator — only when domains are genuinely distinct.

Hierarchical — only when truly complex.

Each step costs more code, more debugging, more latency, more money. Make sure you're buying something with that cost.

ANTI-PATTERNS

Multi-agent theatre.

Building a complex multi-agent system because it sounds cool. Premature decomposition before you understand the domain.

Make things as simple as possible. But not simpler.

Pattern comparison.

PATTERN	LATENCY	COST	COMPLEXITY	BEST FOR
Sequential	Low	Low	Low	Linear pipelines
Parallel	Low	High	Medium	Independent sub-tasks
ReAct	Medium	Medium	Low	Open-ended single domain
Coordinator	Medium	Medium	Medium	Multiple domains
Hierarchical	High	Very High	High	Genuinely complex

A hierarchical run can cost an order of magnitude more than a single ReAct loop.

Production considerations.

TRADE-OFFS TO BALANCE

LATENCY vs. accuracy. Single fast call vs. iterative refinement.

COST vs. quality. Cheap model for the easy 80%, expensive for the hard 20%.

AUTONOMY vs. safety. Full autonomy is fast and risky.

PRODUCTION CHECKLIST

Monitoring — track every call.

Logging — debug failed requests.

Rate limiting — prevent runaway costs.

Fallbacks — handle LLM and tool failures.

Evaluation — measure quality.

Cost tracking — daily, per-user.

Part 03 — summary.

SINGLE-AGENT

ReAct — default. Reason, act, observe, repeat.

DETERMINISTIC

Sequential — fixed pipeline.
Parallel — concurrent fan-out.

DYNAMIC

ReAct — autonomous loop.
Coordinator — router + specialists.
Hierarchical — multi-level decomposition.

SPECIAL

Review & critique — quality gate.
HITL — approval gate.

GOLDEN RULE

Start simple. Add complexity only when it pays for itself.

PRODUCTION

Balance accuracy, speed, cost, and safety. None for free.

Course summary.

PART 01

System architecture.

High-level structure linking requirements to code.

Patterns and trade-offs.

Conway's law: arch mirrors org.

Start simple. Evolve.

PART 02

Agent architecture.

Model + tools + instructions.

Memory — short and long term, RAG.

MCP, tool bloat, mitigation.

Seven layered guardrails.

HITL for high-risk.

PART 03

Orchestration.

Five things.

Architecture *mirrors* the organization. Conway's law applies to everything.

Start simple. Monolith first. Single agent first. Evolve as needed.

Agents = model + tools + instructions. All three.

Safety is layered, not optional. Guardrails plus HITL plus monitoring.

Pattern selection matters. There is no one-size-fits-all.

Questions?

DISCUSSION

Real-world architecture decisions you've faced.

Challenges in building AI agents.

Pattern selection for your projects.

OFFICE HOURS

Wednesdays · 2–4pm · E2-215

PROJECT IDEAS

Multi-agent research assistant.

Intelligent customer support system.

Code review & security audit agent.

Content generation with quality gates.

Thank you.

Slides on Canvas.

CSE115A · SPRING 2026

Scott Davis · okdogulu@ucsc.edu

Non scholae sed vitae discimus.